

Summary

This topic provides a technical synthesis of advanced graph traversal concepts, focusing on Depth First Search (DFS) applications in undirected and directed graphs. Key points include the creation of spanning forests in disconnected graphs, edge classification in directed graphs, and the requirements for Topological Sorting in Directed Acyclic Graphs (DAGs). It also outlines DFS implementation and graph representation methods.

1. DFS in Disconnected and Undirected Graphs

- Traversal Process: Start at an arbitrary unvisited node. Once all reachable nodes are explored, search for remaining unvisited nodes and repeat.
- Resultant Structure: DFS on a disconnected undirected graph produces a Forest of Spanning Trees.
- Connectivity Definition: A graph is connected if a path exists between every pair of nodes. Disconnected graphs naturally result in multiple components.

2. Analyzing Connectivity via Discovery and Finish Times

Case Study 1: Connected Graph

Node	Discovery Time (d)	Finish Time (f)
A	4	20
B	5	15
C	7	14
D	10	12

Conclusion: The nested times indicate all nodes are connected in a single DFS tree.

Case Study 2: Disconnected Graph

Node	Discovery Time (d)	Finish Time (f)
A	1	10
B	2	9

C	6	7
D	13	15

Conclusion: Nodes A, B, C form one component, while D forms another. Disconnected components are evident from non-overlapping finish times.

3. DFS in Directed Graphs and Edge Classification

In directed graphs, DFS edges are classified as follows:

- Tree Edges: Part of the DFS tree or forest.
- Forward Edges: Point from a node to a non-child descendant.
- Back Edges: Point from a node to an ancestor; their presence indicates a cycle.
- Cross Edges: Between nodes with no ancestor/descendant relationship (different branches or trees).

4. Directed Acyclic Graphs (DAG) and Topological Sort

- DAG: A directed graph with no cycles.
- Topological Sorting: Linear ordering of nodes such that for every edge $(U \rightarrow V)$, (U) appears before (V) .

Applications:

- Modeling dependencies (e.g., course prerequisites).
- Transaction serialization in databases.

Preconditions:

- Only possible for DAGs.
- Presence of back edges prevents a valid topological order.

Key Entities:

- Source: Node with in-degree 0.
- Sink: Node with out-degree 0.

5. Algorithmic Implementation and Representation

DFS Pseudo-code Logic:

1. Initialize visited array to 0.
2. Call DFS(source_node).
3. Procedure:
 - Mark current node as visited = 1.
 - For each neighbor W of current node V:
 - If visited[W] == 0, recursively call DFS(W).

Graph Representations:

Representation	Description	Characteristics
Adjacency Matrix	$n \times n$ matrix; $A[i][j] = 1$ if edge exists between i and j	$O(1)$ edge lookup; requires $O(n^2)$ space
Adjacency List	Array of n pointers to linked lists of neighbors	Space-efficient for sparse graphs; neighbor order depends on implementation